UNIT-2

Pandas for Data Handling

What is Pandas?

- Open-source Python library for data manipulation and analysis
- Built on top of NumPy
- Provides fast, flexible, and expressive data structures
- Widely used in Data Science, Machine Learning, and Analytics

Why Pandas?

- Handles structured data efficiently
- Simplifies data cleaning and preparation
- Integrates well with visualization and ML libraries

Pandas vs NumPy:

- NumPy focuses on numerical computation using arrays
- Pandas focuses on data handling and analysis
- Pandas supports heterogeneous data and missing values
- Pandas provides rich file I/O (CSV, Excel, SQL)

Pandas Series:

- One-dimensional labeled data structure
- Each element has an index
- Can store numeric or non-numeric data
- Useful for single-column data

Creating a Pandas Series:

```
import pandas as pd

s = pd.Series([10, 20, 30, 40])
print(s)

s2 = pd.Series([90, 85, 88],
               index=["Math", "Science", "English"])
print(s2)
```

Pandas DataFrame:

- Two-dimensional tabular data structure
- Rows represent observations
- Columns represent features
- Can store multiple data types

Creating a Pandas DataFrame:

```
import pandas as pd

data = {
    "Name": ["Asha", "Ravi", "Neha"],
    "Marks": [85, 92, 78],
    "Passed": [True, True, True]
}

df = pd.DataFrame(data)
print(df)
```

Reading CSV Files:

- CSV is a common data storage format
- Pandas provides read_csv() function
- Automatically creates a DataFrame

```
import pandas as pd

df = pd.read_csv("students.csv")
print(df.head())
```

Writing CSV Files:

- Processed data can be saved to CSV
- Useful for reporting and reuse
- Controlled using to_csv()

- **Why index=False?**
- Prevents writing row index as an extra column

```
df.to_csv("output_students.csv",
          index=False)
```

Inspecting Datasets:

- Understand structure and quality of data
- Identify missing values and data types
- Decide preprocessing steps

```
df.head()
df.info()
df.describe()
```

Inspecting Dataset – Basic Methods:

- Common inspection functions:
- head() – First few rows
- tail() – Last few rows
- info() – Structure and data types
- describe() – Statistical summary

Understanding Dataset Shape and Columns:

- shape – Number of rows and columns
- columns – Column names
- index – Row labels
- **Why it matters:**
- Confirms dataset size
- Helps in feature selection
- Avoids dimension mismatch errors

Selecting Data:

- Select specific columns or rows
- Useful for feature selection
- Reduces data complexity

Selecting Columns:

```
marks = df["Marks"]

subset = df[["Name", "Marks"]]
```

- Single column selection → returns Series
- Multiple column selection → returns DataFrame

Typical Use Cases:

- Feature extraction
- Target variable selection

Selecting Rows using loc & iloc:

```
# Using loc (label-based)
row1 = df.loc[0]

# Using iloc (position-based)
row2 = df.iloc[1]
```

loc vs iloc:

- **loc:**
- Label-based indexing
- Includes both start and end labels
- **iloc:**
- Integer-based indexing
- End index is excluded

Filtering Data:

- Select rows based on conditions
- Helps focus on relevant data
- Improves analysis accuracy

Filtering Data Using Conditions:

```
high_scorers = df[df["Marks"] > 80]
print(high_scorers)
```

- Condition creates a boolean mask
- Only rows satisfying the condition are selected

Filtering with Multiple Conditions:

- Combine conditions using:
  AND (&)
  OR (|)

```python
import pandas as pd

data = {
    "Name": ["Asha", "Ravi", "Neha", "Kiran"],
    "Marks": [85, 92, 78, 88],
    "Passed": [True, True, False, True]
}

df = pd.DataFrame(data)
print(df)
```

```python
filtered_df = df[(df["Marks"] > 80) & (df["Passed"] == True)]
print(filtered_df)
```

Best Practices

- Always inspect data before analysis
- Handle missing values carefully
- Use meaningful column names
- Validate filtered results

Create Dataframe for this:

ID	Color	Size	City	Crop	Yield
1	Red	Small	Delhi	Rice	20
2	Blue	Medium	Mumbai	Wheat	25
3	Green	Large	Delhi	Rice	22
4	Blue	Small	Chandigarh	Maize	18
5	Red	Medium	Mumbai	Wheat	30